Confidential Inference on TrustZone/OP-TEE: Complete Implementation Documentation

RawTFNet audio deepfake detection inside an emulated ARM TrustZone secure world — architecture, full command reference, every challenge encountered with its resolution, and the end-to-end verification chain. MSc Computing thesis PoC, Waleed Asif, ATU Galway.

---

1. Final Result

The trained RawTFNet_small deepfake-detection model runs inside an OP-TEE Trusted Application (TA) on the QEMU Armv8-A TrustZone reference platform. A normal-world client application sends audio-derived features across the secure-world boundary; the TA executes the model's classifier and returns spoof/bonafide logits. The score produced **inside the secure world is identical to the original PyTorch model's output to six decimal places** (–5.621588) on the reference utterance, closing a seven-stage verification chain from trained checkpoint to in-TEE inference.

```
# Normal World console (QEMU guest)
# optee_example_rawtfnet /mnt/host/features.bin
Invoking RawTFNet classifier TA in secure world...
logits: 5.391142 -5.621588
verdict: SPOOF (fake)
```

2. Final Architecture: Partitioned TEE Inference

The deployed design is a **partitioned inference architecture**, in the style established in the TEE machine-learning literature (e.g. DarkneTZ), where a network is split across the trust boundary. The split point is the model's natural structure: `forward() = classifier(front_end(x))`.

```
any audio file (.wav/.flac/.mp3)
| librosa load @16 kHz, repeat-pad to 64,600 samples
v
[NORMAL WORLD, host] front_end (sinc layer + conv blocks, via ONNX Runtime)
| features: (1, 32, 23, 16) = 11,776 float32 (~46 KB)
v
[NORMAL WORLD, guest] client app (optee_example_rawtfnet) -- TEE Client API -->
v
[SECURE WORLD] Trusted Application ba70f032-...-81c9
classifier head (emx-onnx-cgen generated C, weights inline)
| logits[2]
v
[NORMAL WORLD] spoof / bonafide verdict
```

Why partitioned: the generated C for the full model allocates 191 intermediate tensors totalling **319.5 MB** of workspace (largest single buffer 33 MB), measured directly from the generated source. QEMU's virt machine provides a fixed secure RAM region of approximately 16 MB; expanding it requires coordinated patches to QEMU, ARM Trusted Firmware, and OP-TEE's memory maps. The front-end therefore executes in the normal world and the classifier — whose workspace is trivially small — executes confidentially. The measured 319.5 MB requirement is itself a reportable result: it quantifies the secure-memory provisioning a full-model deployment would demand.

Security scope (state this precisely in the thesis): the classifier weights and the final decision computation are confidential and tamper-protected from normal-world software. The front-end feature extractor is not. This is deliberate, measured, and standard practice under TEE memory constraints.

3. Environment Setup (Complete Commands)

3.1 Host: VirtualBox Ubuntu VM, bridged networking

Ubuntu 22.04 VM with a bridged network adapter (stable LAN IP — required later for Asterisk SIP/RTP). 4+ cores, 8+ GB RAM, ~60 GB disk. Two Python contexts are used and must not be mixed: the **system Python** (has the cryptography module OP-TEE's TA signing needs) for all OP-TEE builds, and a **venv "tvm-env"** for all model tooling (onnx, onnxruntime, onnxsim, emx-onnx-cgen, librosa, tvm). Building OP-TEE with tvm-env active breaks TA signing.

```
sudo apt update && sudo apt install -y android-tools-adb android-tools-fastboot \
autoconf automake bc bison build-essential ccache cscope curl device-tree-compiler \
expect flex ftp-upload gdisk libattr1-dev libcap-dev libcap-ng-dev libfdt-dev \
libftdi-dev libglib2.0-dev libhidapi-dev libncurses5-dev libpixman-1-dev \
libslirp-dev libssl-dev libtool make mtools netcat ninja-build \
python3-cryptography python3-pip python3-pyelftools python3-serial \
python3-venv rsync unzip uuid-dev xdg-user-dirs xterm xz-utils zlib1g-dev git \
libgnutls28-dev python3-tomli ffmpeg libsndfile1 openssh-server cmake

mkdir -p ~/bin && curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
chmod a+x ~/bin/repo && echo 'export PATH=~/bin:$PATH' >> ~/.bashrc && source ~/.bashrc
git config --global user.name "Your Name" && git config --global user.email
you@example.com
```

3.2 Build OP-TEE for QEMU Armv8-A (qemu_v8)

```
mkdir -p ~/optee && cd ~/optee
repo init -u https://github.com/OP-TEE/manifest.git -m qemu_v8.xml
repo sync -j4
cd build && make toolchains -j2 && make -j$(nproc)
make run # 'c' + Enter at the (qemu) monitor; login: root
xtest # result: 0 failed of 41,482 subtests / 145 test cases
```

3.3 Model tooling venv

```
python3 -m venv ~/tvm-env && source ~/tvm-env/bin/activate
pip install --upgrade pip
pip install onnx onnxruntime onnxsim onnxoptimizer librosa soundfile
pip install "emx-onnx-cgen[verify]"
# (apache-tvm was additionally used for an independent verification stage)
```

4. Model Conversion Chain (Windows → ONNX → C)

4.1 Minimal inference reference (predict.py, Windows)

A clean load-and-infer path was extracted from the working test_audio.py into predict.py (load_model / load_audio / run_inference), establishing the reference score –5.6216 on utterance LA_E_1000048.flac. Every later stage is checked against this number.

4.2 ONNX export (export_onnx.py, Windows)

```
torch.onnx.export(model, torch.randn(1, 64600), "rawtfnet.onnx",
input_names=["audio"], output_names=["logits"],
opset_version=17, dynamic_axes=None, dynamo=False)
# consolidate weights into ONE file (avoids the .onnx.data external-data split):
m = onnx.load("rawtfnet.onnx", load_external_data=True)
onnx.save_model(m, "rawtfnet.onnx", save_as_external_data=False)
# verification result: PyTorch -5.6216 | ONNX -5.6216 | difference 0.000000
```

4.3 Graph cleaning for embedded codegen (Ubuntu, tvm-env)

```
python3 - <<'EOF'
import onnx
```

```
from onnxsim import simplify
m = onnx.load('rawtfnet.onnx')
m, ok = simplify(m); assert ok
for node in m.graph.node: # strip training-only attribute
if node.op_type == 'BatchNormalization':
a = [x for x in node.attribute if x.name != 'training_mode']
del node.attribute[:]; node.attribute.extend(a)
onnx.save(m, 'rawtfnet_clean.onnx')
EOF
```

4.4 C code generation with emx-onnx-cgen

onnx2c was attempted first and rejected after three successive operator/parser gaps (external-data tensors, the training_mode attribute, Squeeze handling). emx-onnx-cgen (>99% ONNX opset-26 coverage; static, malloc-free-by-design C for embedded targets; built-in verification against ONNX Runtime) succeeded on the first attempt:

```
emx-onnx-cgen verify rawtfnet_clean.onnx --cc gcc
# -> Result: OK (max ULP 7) [generated C vs ONNX Runtime, random inputs]
emx-onnx-cgen compile rawtfnet_clean.onnx rawtfnet.c --large-weight-threshold 0
# -> single self-contained C file, ALL weights inline as static const arrays
# native gate (x86):
gcc -O2 -std=c99 test_main.c rawtfnet.c -lm -o test_native
./test_native input.bin # -> logits: 5.391142 -5.621588 (matches reference)
```

5. Trusted Application and Client Application

The TA/CA pair was derived from the official hello_world example (`cp -r hello_world rawtfnet` inside optee_examples), with UUID ba70f032-7f54-4882-8002-1d7c133d81c9 generated via uuidgen. Files: `ta/rawtfnet_ta.c` (command handler), `ta/rawtfnet_model.c` (generated classifier C), `ta/ta_math.h/.c` (libm subset), `ta/include/rawtfnet_ta.h` (UUID + command ID), `ta/user_ta_header_defines.h` (memory sizes), `ta/sub.mk`, `ta/Makefile` (BINARY = UUID), `host/main.c` (client), plus a corrected CMakeLists.txt.

5.1 The libm problem and ta_math

The TA dev kit ships no math.h or float.h. The generated model uses exactly five libm functions (found by grep): expf, fabsf, fmaxf, logf, sqrtf. These were implemented by hand in ta_math.c (Newton–Raphson sqrtf; range-reduced Taylor expf; atanh-series logf), with FLT_MAX / FLT_EPSILON / INFINITY defined in ta_math.h. The generated file's `#include <math.h>/<float.h>` lines are replaced by `#include "ta_math.h"` via sed. malloc/free must NOT be re-implemented — the dev kit already provides them (bget allocator, sized by TA_DATA_SIZE); re-defining them causes multiple-definition link errors. The exact in-TA score match demonstrates these approximations are adequate for this classifier.

5.2 TA command interface

```
Command TA_RAWTFNET_CMD_INFERENCE (0):
param[0] TEEC_MEMREF_TEMP_INPUT float[11776] features (1,32,23,16)
param[1] TEEC_MEMREF_TEMP_OUTPUT float[2] logits
Handler: validate sizes -> cast shared buffer to (const float(*)[32][23][16])
-> logits initialised to {0,0} -> model(features, logits)
-> memcpy logits to output memref.
Memory: TA_DATA_SIZE = 4 MB, TA_STACK_SIZE = 256 KB,
TA_FLAGS = EXEC_DDR | SINGLE_INSTANCE | MULTI_SESSION
```

5.3 Build and run

```
deactivate # CRITICAL: system Python for TA signing
cd ~/optee/build && make -j$(nproc)
cp ~/features.bin ~/qemu_share/
make run QEMU_VIRTFS_ENABLE=y QEMU_VIRTFS_HOST_DIR=$HOME/qemu_share
# in the (qemu) monitor: c <Enter>; Normal World login: root
mkdir -p /mnt/host && mount -t 9p -o trans=virtio host /mnt/host
optee_example_rawtfnet /mnt/host/features.bin
```

6. The Memory Wall: Investigation Record

Getting the full model to load in the TA failed in instructive ways. The record below is the chronological diagnosis; it is thesis material in its own right because each step produced a measurement.

Observation	Diagnosis / measurement	Outcome
TEEC_OpenSession 0xffff000c; secure log: E/LD populate_segments: sys_map_zi	TA loader cannot map the requested zero-init region. Initially attributed to the model's .bss; native measurement later showed model .bss is only ~258 KB — the real trigger was TA_DATA_SIZE set to 32–48 MB, exceeding QEMU's ~16 MB secure pool by itself.	Any TA_DATA_SIZE above the pool fails at load regardless of the model.
Stage-1 experiment: trivial TA (no model) summing the input	Loaded and ran; returned n=64600 and the correct sum across the world boundary.	Proved the TA/CA path; isolated the fault to model memory.

Observation	Diagnosis / measurement	Outcome
Heap-temporaries regen (--large-temp-threshold huge) + 2–4 MB stack	TA loaded but panicked: user-mode data abort with fault address at sp_el0 — stack overflow; the “heap” temporaries became stack locals.	Approach abandoned.
Static-codegen TA with small heap “ran” and printed varying logits (–0.0/0.0, 4.49, 6.40)	Generated model() mallocs ALL intermediates up front and returns silently if any malloc fails — logits were never written; values were uninitialised stack garbage. Handler now zero-initialises logits.	All pre-partition “scores” were artefacts, not partial inference.
Source-level workspace audit (regex over generated mallocs)	191 tensors, largest 33.0 MB (16×24×21490 floats), TOTAL 319.5 MB. Even ideal producer/consumer reuse needs ≥66 MB concurrently.	Full model cannot fit QEMU's ~16 MB secure region → partition.
Attempted input-window reduction (2 s / 32,000 samples)	Forward pass breaks structurally: classifier max-pool output collapses to size 0 (“Output size is too small”); accuracy also degrades outside 2–4 s per prior experiments.	Input fixed at the native 64,600 samples; live-call windows must match.

7. The Partitioned Solution (Commands)

```
# Windows: split exports + reference features
python export_classifier.py # prints shape (1,32,23,16); logits [[5.391142 -5.6215887]]
python export_frontend.py # front_end-only ONNX for the any-audio pipeline
scp rawtfnet_classifier.onnx rawtfnet_frontend.onnx features.bin
waleed@<VM-IP>:/home/waleed/

# Ubuntu (tvm-env): clean -> verify -> compile -> native gate
<onnxsim + training_mode strip> -> rawtfnet_classifier_clean.onnx
emx-onnx-cgen verify rawtfnet_classifier_clean.onnx --cc gcc
emx-onnx-cgen compile rawtfnet_classifier_clean.onnx rawtfnet_cls.c
--large-weight-threshold 0
gcc -O2 -std=c99 test_cls.c rawtfnet_cls.c -lm -o test_cls
./test_cls features.bin # -> logits: 5.391142 -5.621588 (lossless split)

# TA swap: cp rawtfnet_cls.c -> ta/rawtfnet_model.c ; sed include fixes;
# handler + client updated to 11,776-float input; TA_DATA_SIZE 4 MB; rebuild; run.
```

8. Running Any Audio File (Operational Workflow)

After one-time setup (rawtfnet_frontend.onnx on the VM; prepare_audio.py in the home directory), classifying any audio file is two commands. QEMU stays running between files — the 9p shared folder updates live.

```
# Ubuntu host (tvm-env active) - any .wav/.flac/.mp3:
python3 ~/prepare_audio.py /path/to/any_audio.wav
# loads @16 kHz, repeat-pads to 64,600 samples, runs front_end via ONNX Runtime,
# writes ~/qemu_share/features.bin (asserts shape (1,32,23,16))

# QEMU Normal World (mount once per boot):
mkdir -p /mnt/host && mount -t 9p -o trans=virtio host /mnt/host
optee_example_rawtfnet /mnt/host/features.bin
# -> logits + SPOOF/BONAFIDE verdict computed in the secure world
```

9. Complete Challenge Log

#	Challenge	Root cause	Resolution
1	u-boot build: gnutls/gnutls.h missing	mkeficapsule needs GnuTLS headers not in the standard prerequisite list	apt install libgnutls28-dev; resume make
2	QEMU build: “found no usable tomli”	QEMU's meson build requires the tomli Python module	apt install python3-tomli
3	HTTP 404 during QEMU subproject download	Transient mirror failure	Re-run make; build resumed and self-recovered
4	onnx load: “data … should be stored in rawtfnet.onnx.data”	PyTorch exporter split weights into an external-data sidecar; only the graph file was copied	Re-export and consolidate with onnx.save_model(..., save_as_external_data=False)
5	torch.onnx.export: No module named onnxscript	New default (dynamo) exporter dependency	pip install onnxscript (alternative: dynamo=False)
6	TVM 0.25: relay / tvm.nd / runtime.ndarray all missing	Relay API fully replaced by Relax; array API replaced by tvm.runtime.tensor; docs/tutorials outdated	Introspected installed module (dir(tvm)); ported script to relax + from_onnx + runtime.tensor; documented as toolchain-instability risk
7	librosa FLAC decode failure on VM	libsndfile FLAC path failing; no audioread backend	apt install ffmpeg libsndfile1

#	Challenge	Root cause	Resolution
8	onnx2c: external data / “Unknown attribute training_mode” / “No axes to squeeze away?”	Parser/operator gaps for this architecture (BatchNorm attrs, SE-module Squeeze)	onnxsim + explicit training_mode strip helped; after the third gap, switched code generator to emx-onnx-cgen
9	onnx2c test-suite _Float16 build errors	Host toolchain lacks _Float16 for test cases	Irrelevant to the tool: build only the binary (make onnx2c)
10	TA compile: fatal error math.h not found	TA dev kit provides no libm headers	ta_math.h/.c implementing the 5 required functions; sed the includes
11	TA compile: fmaxf implicit; INFINITY undeclared; malloc implicit	Same restricted environment (no float.h/stdlib declarations reached)	Declarations + INFINITY/FLT_MAX in ta_math.h; #include (dev kit provides it)
12	TA link: multiple definition of malloc/free	Hand-written allocator collided with the dev kit's bget allocator	Deleted own implementations; declare-only
13	TA signing: No module named cryptography	OP-TEE build run inside tvm-env, whose Python lacks cryptography	Rule: always deactivate the venv for OP-TEE builds (system Python)
14	CMake error in examples build	Copied CMakeLists.txt still referenced hello_world targets	Rewrote CMakeLists.txt for optee_example_rawtfnet
15	TA load failure: sys_map_zi / 0xffff000c	TA_DATA_SIZE (32–48 MB) exceeded QEMU's ~16 MB fixed secure region; later: full-model workspace measured at 319.5 MB	Sized-down PoC values; ultimately architectural: partitioned inference
16	TA panic TEE_ERROR_TARGET_DEAD, fault at sp_el0	Heap-threshold regen placed large temporaries on the TA stack → stack overflow	Approach abandoned in favour of measurement + partition
17	Plausible-looking but wrong logits from the TA	model() returns silently on malloc failure; logits were uninitialised stack memory	Zero-initialise logits; treat exact zeros as failure signal; root-cause via source audit
18	Input window cannot be shortened	Classifier max-pool collapses to size 0 below the native window; accuracy band is 2–4 s	Keep 64,600 samples end-to-end; constrains live-call windowing

10. Correctness: The Verification Chain and Its Limits

Stage	Method	Score / result
PyTorch (Windows, reference)	predict.py on LA_E_1000048.flac	–5.6215887
ONNX Runtime	export_onnx.py side-by-side comparison	–5.6216, difference 0.000000
Apache TVM (Relax, LLVM)	independent compiler stack, same input	–5.621588
Generated C vs ONNX Runtime	emx-onnx-cgen verify, random inputs	OK, max ULP 7 (full model)
Native C (x86, full model)	test_native on input.bin	–5.621588
Classifier split (PyTorch)	classifier(front_end(x)) vs full forward	identical: [[5.391142 –5.6215887]]
Native C (classifier only)	test_cls on features.bin	–5.621588
Inside the TA (secure world)	optee_example_rawtfnet, QEMU TrustZone	5.391142 / –5.621588 — exact match

Is this "100%" correct? No software claim reaches 100%, and the thesis should not say so. What the evidence establishes: (1) **pipeline fidelity** — eight independent representations of the model produce the same score on the reference input, and graph-level randomised verification bounds the generated C at 7 ULP from ONNX Runtime; (2) the hand-written expf/logf/sqrtf in the TA are accurate enough that the secure-world score matches to six decimals. What remains open, and should be stated: end-to-end verification used one reference utterance plus randomised graph checks — a batch run of 20–50 ASVspoof eval files through both PyTorch and the TA pipeline, reporting maximum score deviation, would strengthen the claim; pipeline fidelity is distinct from **detection accuracy**, which remains the model's EER as measured in PyTorch; and small numeric differences between the PyTorch and ONNX-Runtime front-ends may shift last-decimal scores on other files without affecting verdicts.

11. Limitations (For the Thesis)

Limitation	Statement
Partial confidentiality	Only the classifier head executes in the TEE; the front-end feature extractor runs in the normal world. Justified quantitatively: full-model in-TEE inference requires 319.5 MB of secure workspace vs ~16 MB available.
Emulated platform	QEMU Armv8-A virt validates the architecture and isolation model, not device silicon; no hardware root of trust, fused keys, or attestation.
Codegen memory model	emx-onnx-cgen allocates all intermediates for the graph lifetime (no buffer reuse) — its correctness-first non-goal; a reuse-planning compiler would shrink the requirement substantially and is future work.
Hand-written math subset	expf/logf/sqrtf implemented manually for the TA; validated empirically by exact score match, not formally.
Single-utterance E2E verification	Graph-level verification is randomised and tight (7 ULP); end-to-end secure-world verification used one reference file. Batch comparison recommended.
Fixed input window	The model structurally requires its native 64,600-sample (~4 s) window; live-call integration must buffer to that length.
Weights compiled into the TA	Retraining requires re-signing the TA; production would use OP-TEE secure storage for weight provisioning (deferred with federated learning).

12. Next Step

Live integration: feed the prepare-audio stage from the Asterisk rolling call buffer (4-second windows at 16 kHz), invoking the TA per window — the operational pipeline in Section 8 is already the shape of that loop.